



TASK

Control Structures – For and While Loops

Visit our website

Introduction

WELCOME TO THE CONTROL STRUCTURES – FOR AND WHILE LOOPS TASK!

In this task, you will explore loop structures to make code more efficient, error-free, and reusable. Loops allow repeated execution of a block of code, shortening and simplifying the code. We will start with the while loop, the simplest type of loop. Then, we will move on to for loops, useful when the number of iterations is known. Finally, we will look at nested loops for solving more complex problems. Understanding loops is crucial for developing efficient algorithms and handling repetitive tasks, enhancing your coding skills, and improving efficiency.

GET INTO THE LOOP OF THINGS

Loops are handy tools that enable programmers to do repetitive tasks with minimal effort. To count from one to 10, we could write the following program:

```
console.log(1);  
console.log(2);  
console.log(3);  
console.log(4);  
console.log(5);  
console.log(6);  
console.log(7);  
console.log(8);  
console.log(9);  
console.log(10);
```

Although the numbers one to 10 **will** be printed by the code above, there are a few problems with this solution:

- **Efficiency:** Repeatedly coding the same statements takes a lot of time.
- **Flexibility:** What if we wanted to change the start number or end number? We would have to go through and change each line of code, adding extra lines of code where they're needed.
- **Scalability:** 10 repetitions are trivial, but what if we wanted 100 or even 100,000 repetitions? The number of lines of code needed would be overwhelming and very tedious for a large number of iterations.

- **Maintenance:** Where there is a large amount of code, the programmer is more likely to make a mistake.
- **Feature:** The number of tasks is fixed and doesn't change at each execution.

Consider the following code:

```
// Initialise the number to start at 0.  
let number = 0;  
  
// Set a condition for the loop to repeat itself until 10 is reached  
while (number < 10) {  
  number++; // Increment number by 1 to ensure the output starts at 1 not 0  
  console.log(number); // Output the count from 1 to 10  
}
```

If we run the program, the same result is produced, but looking at the code, we immediately see the advantage of using a loop. Instead of executing 10 different lines of code, line seven executes 10 times. The original 10 lines of code have been reduced significantly. This has been achieved through the update statement in line six, which adds one to the variable **number** each iteration until **number == 10**. Furthermore, we may change the number 10 to any number we like.

Try it yourself: Replace the 10 with another number. Also, note the first number that is output, and relate that back to the code. How could you rearrange the code, without changing the initial value of **number**, to output the numbers 0 to 9? As a hint, focus on the order of the code lines within the while loop. Consider how moving the **number++** line could impact the output, specifically how the incrementing affects when the number is logged.

WHILE LOOP

A *while loop* is the most general loop statement. The *while statement* repeats its action until the controlling condition becomes false. In other words, the statements indented in the loop repeatedly execute 'while' the condition is true (hence the name). The *while statement* begins with the keyword **while** followed by a boolean expression. The expression is tested before the beginning of each iteration or repetition. If the test evaluates to true, the program passes control to the indented statements in the loop body; if false, control passes to the first statement after the loop body.

Its syntax is as follows:

```
while (boolean expression) {  
    indented statement(s);  
}
```

The following code shows a while statement that sums successive even integers (i.e., $2 + 4 + 6 + 8 + \dots$) until the total is greater than 250. An update statement increments `i` by 2 so that it becomes the next even integer. This is an *event-controlled* loop (as opposed to counter-controlled loops, like the *for loop*) because iterations continue until some noncounter-related condition (event) becomes true and stops the process.

```
// Initialise variable to store the sum  
let sum = 0;  
  
// Initialise even integer to start the sum  
let evenInteger = 2;  
  
// Set the condition for the loop to repeat as long as the sum is <= 250  
while (sum <= 250) {  
    sum += evenInteger; // sum = current value of sum plus the next even  
    integer  
    evenInteger += 2; // Increment the even integer by 2 each time  
    console.log(sum); // Output the sum  
}
```

INFINITE LOOPS

A *while loop* runs the risk of running forever if the condition never becomes false. A loop that never ends is called an *infinite loop*. Creating an infinite loop will mean that your program will run indefinitely while never in fact moving to any code below the *while loop* – not a desirable outcome! Make sure that your loop condition eventually becomes false and that your loop is exited.

Consider the first example we discussed:

```
let number = 0;  
  
while (number < 10) {  
    number++;  
    console.log(number);  
}
```

In any loop, the following three steps are usually used:

1. **Declare a counter/control variable.** The code above does this when it says `number = 0;`. This creates a variable called `number` that contains the value zero.
2. **Increase the counter/control variable in the loop.** In the loop above, this is done with the instruction `number++;` which increases the value of `number` by one (*increments the number*) with each pass of the loop.
3. **Specify a condition to control when the loop ends.** The condition of the *while loop* above is `number < 10`. This loop will carry on executing as long as the `number` is less than ten. This loop will, therefore, execute 10 times. Would there be a difference in the output if the control variable, `number`, was incremented after the `console.log` statement? If so, what would the difference be?

DO WHILE LOOP

The *do while loop* structure has the same functionality as the *while loop*, with the exception of being guaranteed to iterate at least once (because the condition is only checked at the end). This is useful if you want your program to do something before variable evaluation actually begins.

Considering the code below, how many times will the statement 'I have run at least once' be output? What will the value of that output be?

```
// Initialise the variable with a value of -10
let counter = -10;

// Output message until the condition is met
do {
  console.log("I have run at least once!");
  counter++; // Increment the counter by 1
} while (counter <= 1); // Loop will repeat as long as the counter is <= 1

// Output the value of the counter once the loop ends
console.log("The result of the counter is " + counter);
```

Now it is time to move on to for loops. For loops are particularly useful when you know in advance how many times you need to execute a block of code. They provide a concise way to iterate over a range of values, making your code more efficient and easier to read.

FOR LOOP

The *for* loop has very similar functionality to a *while* loop. You will notice that the *for* loop in the example below actually does all the same things as the first *while* loop example we looked at previously, just in a different format.

```
// Iterate through the loop 10 times
for (let i = 1; i <= 10; i++) {
  // Output the value of the variable after each iteration
  console.log(i);
}
```

Compare the *for* loop above with the first *while* loop we looked at previously, and notice the three steps they both have in common:

1. **Declare a counter/control variable.** The code below does this when it says `i = 1`; This creates a variable called `i` that contains the value 1. Something similar was done with the *while* loop with the line `number = 0`;
2. **Increase the counter/control variable in the loop.** In the *for* loop, this is done with the instruction `i++` which increases `i` by one with each pass of the loop. Similarly, the *while* loop did this with the code `number++`.
3. **Specify a condition to control when the loop will end.** The condition of the *for* loop is `i <= 10`. This loop will carry on executing as long as `i` is less than or equal to 10. This loop will, therefore, execute 10 times.

While the variable `i` (an integer) is in the range of 1 to 10, the indented code in the loop's body will execute. In the first iteration, `i` is 1 and 1 is printed. In the second iteration, `i` is 2 and 2 is printed, and so on until `i` reaches 10. Once `i` is no longer in the range 1 to 10, the loop stops executing. `i` is called the index variable, indicating the current iteration.

You can also use an *if* statement within loops! The code in this example will only print the numbers 6, 7, 8, and 9 because numbers less than or equal to 5 are filtered out.

```
// Iterate through the loop less than 10 times
for (i = 1; i < 10; i++) {
  // Set condition to check if i is greater than 5
  if (i > 5) {
    // If the value is greater than 5, output the result.
    console.log(i);
  }
}
```

BREAK STATEMENT

A loop can also contain a **break** statement. Within a loop body, a break statement causes an immediate exit from the loop to the first statement after the loop body. The break allows for an exit at any intermediate statement in the loop such as in the example below:

```
// Iterate through the loop less than 10 times
for (let i = 1; i < 10; i++) {
  // If the counter reaches 5, the loop breaks
  if (i === 5) {
    // Exit the loop
    break;
  }

  // Output the result at the point when the loop breaks
  console.log("Counter: " + i);
}
```

Using a **break** statement to exit a loop has important applications. For example, when reading data from a file, the loop's iterations depend on the amount of data. When the data runs out, a **break** statement allows the loop to exit cleanly upon reaching the end-of-file condition.



Extra resource

and iteration, see [here](#).

Remember to make sure you use the correct syntax rules for each statement as you code using loops. This will ensure that your code not only executes without any unexpected errors, but is also easily readable and maintainable. For further insight into syntax rules in the context of using loops

WHICH LOOP TO CHOOSE?

How do we know which looping structure – the *for loop* or the *while loop* – is more appropriate for a given problem? The answer lies with the kind of problem we are facing. After all, both these loops have the four components that were introduced to you, namely:

- Initialisation of the control variable
- Setting of a termination condition
- Updating of the control variable
- Repetition of the loop body

A **while** loop is used when we don't know how many times the loop will run, as the loop's logic determines when to exit. A **for** loop is used when the number of iterations is known in advance.

For example, use a **for** loop to create a table with 10 rows comparing rand (R) amounts with dollar (\$) amounts, as it runs exactly 10 times. Use a **while** loop to repeatedly divide a number by 2 until it's less than 1, since the number of iterations is not known beforehand.

NESTED LOOPS

A *nested loop* is simply a loop within a loop. Each time the outer loop is executed, the inner loop is executed right from the start. That means that *all the iterations of the inner loop are executed with each iteration of the outer loop*.

The syntax for a nested *for loop* in another *for loop* is as follows:

```
for (let iteratingVariable = 0; iteratingVariable < sequence.length;
iteratingVariable++) {
  for (let innerVariable = 0; innerVariable < sequence.length;
innerVariable++) {
    nested loop statements(s);
  }
  outer loop statements(s);
}
```


The syntax for a nested *while loop* in another *while loop* is as follows:

```
while (condition) {
  while (condition) {
    nested loop statement(s);
  }
  outer loop statement(s);
}
```

You can put any type of loop inside of any other kind of loop. For example, a *for loop* can be inside a *while loop*, or vice versa:

```
for (let iteratingVariable = 0; iteratingVariable < sequence.length;
iteratingVariable++) {
  while (condition) {
    indented statement(s);
  }
  indented statements(s);
}
```

The following program shows the potential of a nested loop. We will walk through what this program does in the next section, so if you are confused at this point do not panic – just read through the code slowly and try to follow what happens.

```
// Outer for Loop for first multiplier
for (let firstNumber = 1; firstNumber < 6; firstNumber++) {
  // Inner for Loop for second multiplier
  for (let secondNumber = 1; secondNumber < 6; secondNumber++) {
    // Return product of first multiplier and second multiplier
    console.log(
      String(firstNumber) +
        "*" +
        String(secondNumber) +
        "=" +
        String(firstNumber * secondNumber)
    );
  }
  // Return empty string between calculations of different multipliers
  console.log("");
}
```

When the above code is executed, it produces the following result:

```
1 * 1 = 1
1 * 2 = 2
1 * 3 = 3
1 * 4 = 4
1 * 5 = 5

2 * 1 = 2
2 * 2 = 4
2 * 3 = 6
2 * 4 = 8
2 * 5 = 10

3 * 1 = 3
3 * 2 = 6
3 * 3 = 9
3 * 4 = 12
3 * 5 = 15
4 * 1 = 4
4 * 2 = 8
4 * 3 = 12
4 * 4 = 16
4 * 5 = 20

5 * 1 = 5
5 * 2 = 10
5 * 3 = 15
5 * 4 = 20
5 * 5 = 25
```

If you are at all confused by nested loops, use a trace table to trace exactly what is happening within the code as each statement executes. We will look at this tool in more detail in the next section.

TRACE TABLES

As you write more complex code, trace tables can help ensure your logic is correct. A trace table tracks variable values for each iteration, which is especially useful for nested loops. Let's revisit the previous code example and create a trace table:

firstNumber	secondNumber	firstNumber * secondNumber
1	1	1
	2	2
	3	3
	4	4
	5	5
2	1	2
	2	4
	3	6
	4	8
	5	10
3	1	3
	2	6
	3	9
	4	12
	5	15
4	1	4
	2	8
	3	12
	4	16
	5	20
5	1	5
	2	10
	3	15
	4	20
	5	25

As you can see, because of the nature of nested loops, the inner loop iterates 5 times before the outer loop is iterated again. That means that the **firstNumber** will remain the same value while the **secondNumber** loops through all its iterations. Then, once the outer loop iterates and its value finally increases, the inner loop restarts with another 5 iterations. This can be seen in the trace table, where the **secondNumber** cycles from 1–5 for each value of the **firstNumber**. Practice drawing trace tables for your upcoming tasks to assist you with the logic – they can be very helpful when coding gets tricky!

Instructions

Read and run the accompanying example files provided before doing the Practical Tasks to become more comfortable with the concepts covered.



Practical task 1

Follow these steps:

- Note: For this task, you will need to create an HTML file to get input from a user. Refer to your previous learning material for a refresher if needed.
- Create a new JavaScript file called **swapping.js**.
- Write a program that asks the user to enter a number of at least three digits.
- Make use of a *for loop* to swap the second digit and last digit in the number.
- Output the original number and the new number.
- *Hint: You can treat the number as a string and use `.split("")` to turn it into an array of digits. Use a **for** loop to swap the second and last digits, then use `.join("")` to rebuild the number.*



Practical task 2

Follow these steps:

- Note: For this task, you will need to create an HTML file to get input from a user. Refer to your previous learning material for a refresher if needed.
- Create a JavaScript file called **palindrome.js**.
- Write a program that asks the user to enter a word.
- Using a *while loop* the program must determine whether or not the word is a palindrome, i.e., whether it reads the same forwards and backwards, e.g., the word 'racecar'.
- Output the given word and whether or not it is a palindrome.

- E.g., “racecar is a palindrome” OR “car is not a palindrome”
- *Hint: Use a **while loop** to build a reversed version of the word by adding characters from the end to the start. Then compare the original word to the reversed one.*

Important: Be sure to upload all files required for the task submission inside your task folder and then click "Request review" on your dashboard.



Share your thoughts

Please take some time to complete this short feedback **form** to help us ensure we provide you with the best possible learning experience.
